Task-12:- Requirement Analysis

1. Introduction

1.1 Purpose

The purpose of this document is to define the requirements for implementing a **monitoring dashboard and alerting system for the Bookstore API**. This document outlines the functional and non-functional requirements needed to monitor API performance, track error rates, analyze HTTP status code distributions, and configure alerts for abnormal API behavior.

The intended audience for this document includes:

- DevOps Engineers
- Operations Team
- Software Developers
- Project Managers
- System Administrators

This document provides a structured understanding of the monitoring system that will be implemented to improve the visibility and reliability of the Bookstore API.

1.2 Scope

The scope of this project includes the implementation of a **monitoring dashboard and alert configuration for the Bookstore API** using a monitoring platform such as **AWS CloudWatch**.

The monitoring system will include:

- Tracking total API request count
- Monitoring HTTP response status codes
- Measuring API error rates
- Visualizing API traffic trends
- Creating alerts for abnormal API behavior
- Designing a clean and readable monitoring dashboard

The project will **not include**:

- Modification of the API source code
- Deployment of the Bookstore API

- Development of new API endpoints
- Database-level monitoring

The focus of this project is strictly on **observability and monitoring of the existing API infrastructure**.

1.3 Definitions, Acronyms, and Abbreviations

Term	Description
API	Application Programming Interface
HTTP	HyperText Transfer Protocol
AWS	Amazon Web Services
CloudWatch	AWS monitoring and observability service
DevOps	Development and Operations practices for automation and monitoring
2xx	HTTP status codes indicating successful responses
4xx	HTTP status codes indicating client errors
5xx	HTTP status codes indicating server errors
Error Rate	Percentage of failed requests compared to total requests
Dashboard	Visual display of metrics and monitoring data

1.4 References

The following resources were referenced while preparing this document:

- AWS CloudWatch Documentation
  https://docs.aws.amazon.com/cloudwatch/
- HTTP Status Code Documentation
  HTTP response status codes - HTTP | MDN
- AWS Monitoring Best Practices
  APM Tool - Amazon CloudWatch - AWS
- DevOps Monitoring Concepts

1.5 Overview

This document provides a structured overview of the monitoring solution required for the Bookstore API.

The document is organized into the following sections:

1. Introduction – Explains the purpose and scope of the project
2. Project Overview – Describes project objectives, goals, and constraints
3. Functional Requirements – Defines monitoring capabilities and use cases
4. Non-Functional Requirements – Describes performance, security, usability, and reliability needs
5. Assumptions and Dependencies – Lists assumptions and dependencies
6. Risk Analysis – Identifies possible risks and mitigation strategies
7. Glossary – Defines technical terms used in the document
8. Appendices – Includes additional diagrams or information
9. Customer Acknowledgement – Confirmation from stakeholders

---

2. Project Overview

2.1 Project Objectives

The main objective of this project is to implement a **monitoring dashboard and alerting system for the Bookstore API** to improve operational visibility and system reliability.

Key objectives include:

- Monitoring API traffic and request counts
- Tracking HTTP status codes returned by the API
- Identifying increases in API error rates

- Visualizing response trends over time
- Configuring alerts for abnormal API behavior
- Improving incident detection and troubleshooting speed

2.2 Business Goals

The project aims to achieve the following business goals:

- Improve reliability and stability of the Bookstore API
- Enable early detection of system failures
- Reduce downtime and service disruptions
- Improve troubleshooting and debugging capabilities
- Provide real-time operational insights for the operations team

By implementing effective monitoring, the organization can maintain high system availability and improve user experience.

2.3 Background

ABC Technologies has deployed the **Bookstore API** to handle application requests related to bookstore services such as retrieving books, processing user requests, and managing data.

However, the operations team currently lacks proper monitoring tools to track API performance and failures.

Without proper monitoring:

- API failures may go unnoticed
- Error patterns cannot be analyzed
- Troubleshooting becomes difficult
- System reliability may decrease

To address these challenges, the company requires a **monitoring dashboard and alerting system** that provides real-time visibility into API behavior.

2.4 Constraints

The project may face the following constraints:

Time Constraint

The monitoring solution must be implemented within a limited timeframe.

Technology Constraint

The monitoring solution must use available tools such as **AWS CloudWatch or the selected monitoring platform**.

Resource Constraint

The implementation will be performed by a **DevOps intern**, which limits the scope of complexity.

Infrastructure Constraint

Monitoring must be integrated with the existing API infrastructure without modifying the core API application.

---

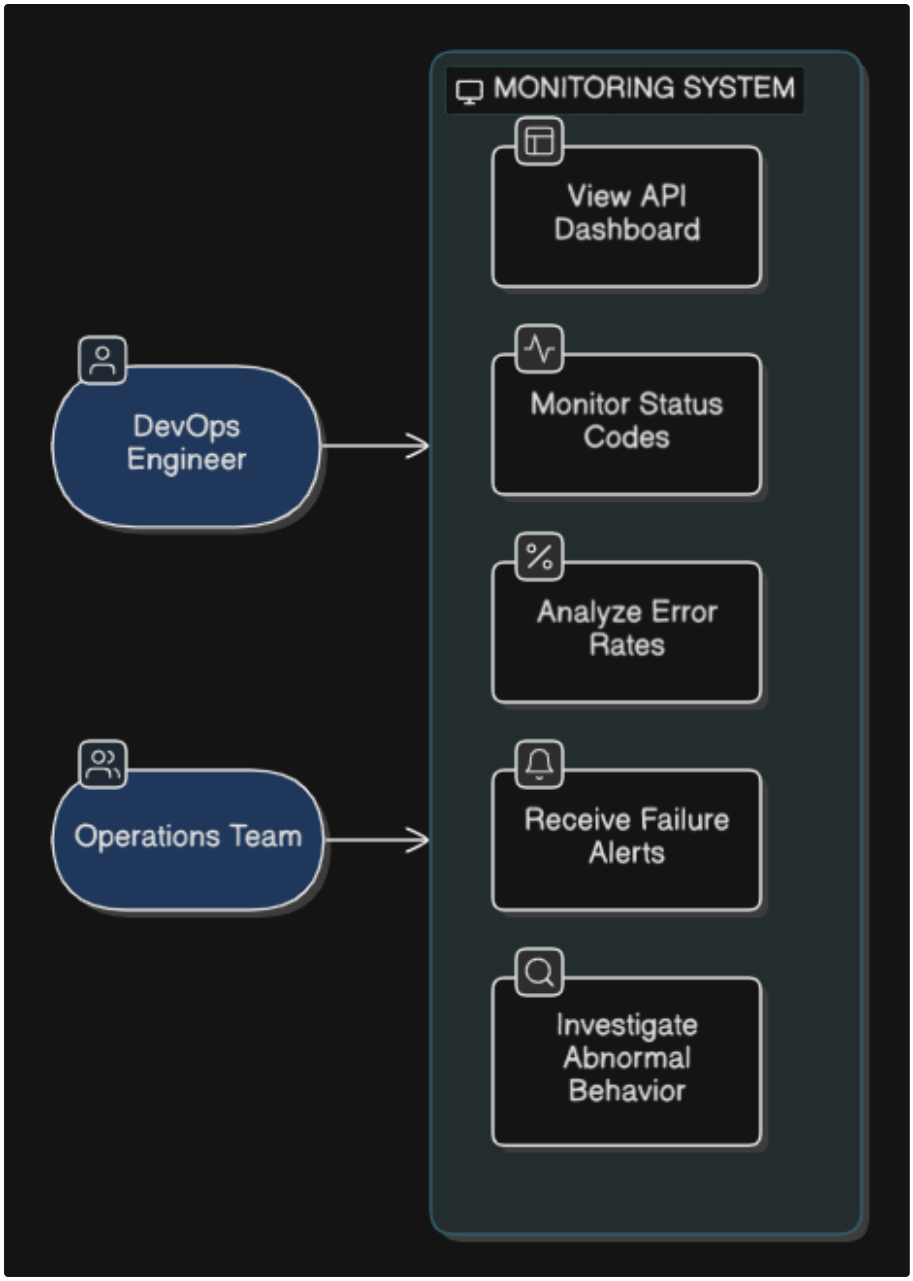
3. Functional Requirements

3.1 Use Case Diagram

The use case diagram represents the interaction between the **DevOps Engineer / Operations Team and the monitoring system**.

The diagram includes the following activities:

- View API monitoring dashboard
- Monitor status codes
- Analyze API error rates
- Receive alerts for API failures
- Investigate abnormal API behavior



3.2 Use Cases

3.2.1 Use Case 1: View API Monitoring Dashboard

Description:
Allows the operations team to monitor API activity using a dashboard that displays request counts, error rates, and status code distribution.

Actors:
Operations Team, DevOps Engineer

Preconditions:

- Monitoring system must be configured
- API logs or metrics must be available

Postconditions:

- Users can view real-time API metrics
- API performance data becomes visible

Flow of Events:

Step 1: User opens the monitoring dashboard
Step 2: Dashboard loads API metrics
Step 3: User views request trends and status code distribution

Alternative Flows:

- Dashboard fails to load due to monitoring service failure
- Metrics are temporarily unavailable

3.2.2 Use Case 2: Monitor API Error Rates

Description:
Allows the operations team to track the percentage of failed API requests.

Actors:
Operations Team

Preconditions:

- API monitoring system must collect response data

Postconditions:

- Error rate trends become visible

Flow of Events:

Step 1: Dashboard collects API response data
Step 2: Error rate is calculated
Step 3: Error trend is displayed using charts

Alternative Flows:

- Monitoring service fails to collect logs
- Data visualization fails to update

3.2.3 Use Case 3: Receive API Failure Alerts

Description:

The monitoring system sends alerts when abnormal API behavior occurs.

Actors:

Operations Team

Preconditions:

- Alert thresholds must be configured

Postconditions:

- Alert notifications are triggered

Flow of Events:

Step 1: Monitoring system evaluates API metrics
Step 2: Threshold is exceeded
Step 3: Alert notification is triggered

Alternative Flows:

- Alert configuration fails
- Monitoring service temporarily unavailable

---

4. Non-Functional Requirements

4.1 Performance Requirements

The monitoring system must:

- Update metrics in near real-time
- Handle increasing API traffic
- Display dashboard metrics quickly
- Maintain smooth visualization performance

4.2 Security Requirements

The monitoring system must ensure:

- Secure access to monitoring dashboards
- Restricted access to authorized users

- Secure handling of API logs and metrics

Access control should be implemented using **AWS IAM policies**.

4.3 Usability Requirements

The dashboard should be designed for easy understanding.

Requirements include:

- Clear panel titles
- Organized dashboard layout
- Easy-to-read graphs and charts
- Logical grouping of monitoring metrics

4.4 Reliability Requirements

The monitoring system must:

- Maintain continuous monitoring of the API
- Ensure high availability of dashboards
- Trigger alerts reliably when thresholds are exceeded

4.5 Compliance Requirements

The monitoring solution must comply with:

- Cloud monitoring best practices
- Organizational monitoring policies
- AWS monitoring and logging standards

---

5. Assumptions and Dependencies

5.1 Assumptions

The following assumptions are made during requirement analysis:

- The Bookstore API is already deployed
- API logs or metrics are available for monitoring
- AWS CloudWatch or a similar monitoring platform is accessible

- The operations team will regularly review the monitoring dashboard

---

5.2 Dependencies

The monitoring solution depends on:

- API infrastructure availability
- AWS CloudWatch services
- Availability of API logs and metrics
- Network connectivity

---

6. Risk Analysis

6.1 Risk Identification

Risk	Description
Monitoring misconfiguration	Incorrect metric configuration may lead to inaccurate data
Alert threshold issues	Alerts may trigger too frequently or not at all
Dashboard overload	Too many metrics may reduce readability
Log collection failure	API logs may fail to reach the monitoring system

---

6.2 Risk Mitigation

Risk	Mitigation Strategy
Monitoring misconfiguration	Verify metric settings and validate dashboard
Alert threshold issues	Define proper alert thresholds
Dashboard overload	Keep dashboard simple and organized

Log collection failure	Implement log monitoring checks

---

7. Glossary

Term	Definition
Monitoring	Observing system performance and behavior
Error Rate	Percentage of failed API requests
Status Code	HTTP response code returned by an API
Alert	Notification triggered when a metric exceeds a threshold
Dashboard	Visual display of system metrics

---

8. Appendices

Appendices may include supporting materials such as:

- Monitoring architecture diagram
- Dashboard screenshots
- Alert configuration screenshots

---

9. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: _____